

torch.randn#

<https://pytorch.org/docs/stable/generated/torch.randn.html>

Description:

Returns a tensor filled with random numbers from a normal distribution with mean 0 and variance 1 (also called the standard normal distribution). For complex dtypes, the tensor is i.i.d. sampled from a complex normal distribution with zero mean and unit variance as This is equivalent to separately sampling the real (`Re`) and imaginary (`Im`) part of `out_i` as The shape of the tensor is defined by the variable argument `size`. `size` (`int...`) – a sequence of integers defining the shape of the output tensor. Can be a variable number of arguments or a collection like a list or tuple.

Function Signature

```
torch.randn(*size, *, generator=None, out=None, dtype=None, layout=torch.strided, device=None, requires_grad=False, pin_memory=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

generator (torch.Generator, optional) – a pseudorandom number generator for sampling out (Tensor, optional) – the output tensor. dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see torch.set_default_dtype()). layout (torch.layout, optional) – the desired layout of returned Tensor. Default: torch.strided. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False. pin_memory (bool, optional) – If set, returne

Code Examples

```
>>> torch.randn(4)
tensor([-2.1436,  0.9966,  2.3426, -0.6366])
>>> torch.randn(2, 3)
tensor([[ 1.5954,  2.8929, -1.0923],
        [ 1.1719, -0.4709, -0.1996]])
```

Detailed Documentation

Documentation

Returns a tensor filled with random numbers from a normal distribution with mean 0 and variance 1 (also called the standard normal distribution).

For complex dtypes, the tensor is i.i.d. sampled from a complex normal distribution with zero mean and unit variance as

This is equivalent to separately sampling the real (Re) and imaginary (Im) part of out_i as

The shape of the tensor is defined by the variable argument size.

size (int...) – a sequence of integers defining the shape of the output tensor. Can be a variable number of arguments or a collection like a list or tuple.

generator (torch.Generator, optional) – a pseudorandom number generator for sampling

out (Tensor, optional) – the output tensor.

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`).

layout (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`.

device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

requires_grad (bool, optional) – If autograd should record operations on the returned

tensor. Default: False.

pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.